



Prueba de Desempeño: AI Backend Engineer

Título:

Asistente Inteligente de Admisiones Universitarias con RAG en Python

Caso de uso (Épica):

Como AI Engineer junior, recibes el encargo de la oficina de admisiones de una universidad tecnológica que está saturada respondiendo las mismas preguntas todos los días por Telegram, correo o su página web: horarios, precios, niveles, inscripciones, certificaciones y modalidades. Tu reto es construir un asistente inteligente que responda automáticamente las consultas frecuentes basándose únicamente en los documentos del negocio, escale al equipo humano cuando no pueda responder, mantenga el tono de marca y sea eficiente en costos de API.

Requisitos Funcionales

1. Recepción de consultas

El sistema debe recibir preguntas por al menos una vía: bot de Telegram, webhook HTTP o formulario web simple. Para este caso, utilizarás un endpoint construido en FastAPI o Flask.

2. Integración con APIs de IA

Procesar consultas usando OpenAI API (o el proveedor LLM de tu elección) en Python, con API Keys en variables de entorno y manejo básico de errores.

3. RAG sobre documentos del negocio



Cargar al menos 3 documentos del negocio, aplicar *chunking* con *overlap*, almacenar en una base vectorial (ej. ChromaDB, FAISS o Qdrant) y responder únicamente con información de esos documentos. Si la pregunta está fuera de *scope*, escalar a un humano.

4. Automatización con n8n

Construir un pipeline en Python puro o utilizando LangChain/LlamaIndex que reemplace el flujo de n8n, conectando el *trigger* (la entrada de la API), la búsqueda semántica, la generación de la respuesta y la lógica condicional para el escalamiento a humanos.

5. Prompt engineering

Crear un *system prompt* con rol, personalidad y restricciones, incluyendo al menos 3 *few-shot examples*. Incluir instrucciones precisas para evitar alucinaciones y configurar la temperatura ajustada al caso de uso.

6. Puntos Extra (Max. 10)

Implementar skills personalizados o MCP para extender el agente

Métricas básicas: consultas procesadas, costo total, tasa de escalamiento

Deploy en URL pública (Railway, Vercel, Render o Lovable)

Caché de respuestas frecuentes para reducir costos

Entregables Esperados

Toda la entrega debe ir en un archivo .zip con los siguientes componentes:



Componente	Descripción	Verificación
Backend en Python	Código modular de FastAPI/ Flask con orquestación del LLM y base vectorial.	Se levanta el servidor localmente y procesa <i>requests</i> sin fallos.
Base vectorial poblada	Script de ingesta o base exportada con los 3+ documentos del negocio cargados. PDF	Las preguntas devuelven <i>chunks</i> relevantes. PDF
Canal de entrada	Telegram bot, webhook o formulario web. PDF	Funcionando. Se envía mensaje y el sistema responde. PDF
README.md en inglés	Setup, arquitectura, variables de entorno, ejemplos de uso. PDF	Se puede ejecutar el proyecto siguiendo el README. PDF
<code>requirements.txt</code> y <code>.env.example</code>	Dependencias del proyecto y todas las variables necesarias documentadas. PDF	<code>pip install -r requirements.txt</code> funciona; ninguna <i>key hardcoded</i> en el código. PDF

Criterios de Aceptación

Responde consultas sin errores críticos

Las respuestas se basan únicamente en los documentos del negocio

Escala correctamente cuando la pregunta está fuera de scope

API Keys nunca hardcodedas en el código

Se puede ejecutar siguiendo el README

Todo el entregable en inglés